

Name: Miriam Wepiya Gale

ID: 46582028

Cohort: B

=====

Exercise: 7.24

SQL Statement (Creating database):

CREATE DATABASE Projects;

Output:

Query OK, 1 row affected (0.002 sec)

SQL Statement(Using the Database):

USE Projects;

Output:

Database changed

SQL Statement(Creating Department table):

CREATE TABLE Department(deptNo INT PRIMARY KEY, deptName VARCHAR(50)
NOT NULL,mgrEmpNo INT);

Output:

Query OK, 0 rows affected (0.008 sec)

SQL Statement(Creating Employee table):

Constraint(a): sex must be one of the single characters 'M'or 'F'

Constraint(b): position must be one of 'Manager', 'Team Leader', 'Analyst' or 'Software Developer'

SQL Statement:

```
CREATE TABLE Employee(empNo INT PRIMARY KEY, fName VARCHAR(20) NOT NULL, lName VARCHAR(50) NOT NULL, address VARCHAR(80) NOT NULL, DOB DATE NOT NULL, sex CHAR(1) NOT NULL CHECK(sex IN('M','F')), position VARCHAR (50) NOT NULL CHECK(position IN('Manager', 'Team Leader', 'Analyst', 'Software Developer')), deptNo INT NOT NULL, FOREIGN KEY(deptNo) REFERENCES Department(deptNo));
```

Output:

Query OK, 0 rows affected (0.022 sec)

SQL Statement:

```
ALTER TABLE Department ADD CONSTRAINT FK_Department_Manager FOREIGN KEY (mgrEmpNo) REFERENCES Employee(empNo);
```

Output:

Query OK, 0 rows affected (0.040 sec)

Records: 0 Duplicates: 0 Warnings: 0

SQL Statement(Creating Project table):

```
CREATE TABLE Project( projNo INT PRIMARY KEY, projName VARCHAR(50) NOT NULL, deptNo INT NOT NULL, FOREIGN KEY(deptNo) REFERENCES Department(deptNo));
```

Output:

Query OK, 0 rows affected (0.020 sec)

Constraint(c): hoursWorked must be an integer between 0 and 40

SQL Statement:

```
CREATE TABLE WorksOn(empNo INT,projNo INT,dateWorked DATE NOT NULL, hoursWorked INT NOT NULL CHECK(hoursWorked BETWEEN 0 AND 40), PRIMARY
```

```
KEY(empNo, projNo, dateWorked), FOREIGN KEY(empNo) REFERENCES
Employee(empNo),FOREIGN KEY(projNo) REFERENCES Project(projNo));
```

Output:

Query OK, 0 rows affected (0.020 sec)

```
=====
=====
```

Populating the Tables

SQL Statement(Department):

```
INSERT INTO Department(deptNo, deptName, mgrEmpNo)
VALUES (1, 'IT', NULL), (2, 'Finance', NULL),(3, 'HR', NULL);
```

Output:

Query OK, 3 rows affected (0.029 sec)

Records: 3 Duplicates: 0 Warnings: 0

SQL Statement(Employee):

```
INSERT INTO Employee (empNo, fName, lName, address, DOB, sex, position, deptNo)
VALUES (101, 'Ama', 'Mensah', 'Accra', '1990-05-10', 'F', 'Manager', 1), (102, 'Kwame',
'Asante', 'Kumasi', '1992-07-15', 'M', 'Analyst', 1), (103, 'Akosua', 'Owusu', 'Cape Coast',
'1994-03-20', 'F', 'Team Leader', 2), (104, 'Kojo', 'Boateng', 'Takoradi', '1993-11-12', 'M',
'Software Developer', 3), (105, 'Abena', 'Agyeman', 'Tema', '1991-09-08', 'F', 'Manager', 2);
```

Output:

Query OK, 5 rows affected (0.017 sec)

Records: 5 Duplicates: 0 Warnings: 0

```
=====
=====
```

Setting the mgrNo in Department Table since Employee table now exists

SQL Statement:

```
UPDATE Department SET mgrEmpNo = 101 WHERE deptNo = 1;
```

Output:

Query OK, 1 row affected (0.017 sec)

Rows matched: 1 Changed: 1 Warnings: 0

SQL Statement:

```
UPDATE Department SET mgrEmpNo = 105 WHERE deptNo = 2;
```

Output:

Query OK, 1 row affected (0.012 sec)

Rows matched: 1 Changed: 1 Warnings: 0

SQL Statement:

```
UPDATE Department SET mgrEmpNo = 104 WHERE deptNo = 3;
```

Output:

Query OK, 1 row affected (0.013 sec)

Rows matched: 1 Changed: 1 Warnings: 0

=====

Populating the Tables

SQL Statement(Project):

```
INSERT INTO Project(projNo, projName, deptNo) VALUES (201, 'Payroll System', 2), (202, 'Website Upgrade', 1), (203, 'Recruitment Portal', 3);
```

Output:

Query OK, 3 rows affected (0.022 sec)

Records: 3 Duplicates: 0 Warnings: 0

SQL Statement(WorksOn):

```
INSERT INTO WorksOn (empNo, projNo, dateWorked, hoursWorked) VALUES (101, 202, '2025-06-01', 20), (102, 202, '2025-06-02', 15), (103, 201, '2025-06-03', 25), (104, 203, '2025-06-04', 18), (105, 201, '2025-06-05', 30);
```

Output:

Query OK, 5 rows affected (0.005 sec)

Records: 5 Duplicates: 0 Warnings: 0

```
=====
```

Exercise 7.25

SQL Statement:

```
CREATE VIEW FemaleManagedProjects AS SELECT p.projNo, p.projName, p.deptNo  
FROM Project p JOIN Department d ON p.deptNo = d.deptNo JOIN Employee e ON  
d.mgrEmpNo = e.empNo WHERE e.sex = 'F' ORDER BY p.projNo;
```

Output:

Query OK, 0 rows affected (0.016 sec)

```
=====
```

Exercise 7.26

SQL Statement:

```
CREATE VIEW EmployeeProjectHours AS SELECT e.empNO, e.fName, e.lName,  
p.projName, w.hoursWorked FROM Employee e JOIN WorksOn w ON e.empNo = w.empNo  
JOIN Project p on w.projNo = p.projNo;
```

Output:

Query OK, 0 rows affected (0.016 sec)

Exercise 7.27

- a. Selects all the tuples and attributes from EmpProject View
 - b. Displays the values projNo column from EmpProject where the projNo(project Number) is 'SCCS'.
 - c. Counts the number of rows in EmpProject where empNo is 'E1'
 - d. Displays the values of empNo and totalHours from the EmpProject View and it should group the results by the empNo
-
-

Exercise 7.28

* Maintaining the materialized view

i. When a new row is inserted: If a new row inserted in the Part table has a partCost greater than £1000, its partNo should be added to the materialised view but if it is less or £1000, the materialised view will not be affected.

ii. When a row is deleted:

If the deleted row from the Part table has a partCost greater than £1000, its respective partNo should be removed from the materialised view because it no longer exists in the Part table. If the deleted row has a partCost of £1000 or less, the materialised view will not be affected.

iii. When the cost of a part is updated:

If the updated partCost changes from £1000 or less to more than £1000, the partNo should be added to the materialized view because it now satisfies the condition $\text{partCost} > 1000$. If the updated partCost changes from more than £1000 to £1000 or less, the partNo should be removed from the materialized view because it no longer satisfies the condition.

* Maintaining the view without accessing the base table

The materialized view can be maintained without accessing the Part table when the changed row already tells us whether the partCost is greater than £1000 or not. This is enough to decide whether the partNo should be added to or removed from the materialized view.